Hierarchical Deep Reinforcement Learning for Joint Service Caching and Computation Offloading in Mobile Edge-Cloud Computing

Chuan Sun, Xiuhua Li, *Member, IEEE*, Chenyang Wang, *Member, IEEE*, Qiang He, *Senior Member, IEEE*, Xiaofei Wang, *Senior Member, IEEE*, and Victor C. M. Leung, *Life Fellow, IEEE*

***Abstract*—Mobile edge-cloud computing networks can provide distributed, hierarchical, and fine-grained resources, and have become a major goal for future high-performance computing networks. The key is how to jointly optimize service caching and computation offloading. However, the joint service caching and computation offloading problem faces three significant challenges of dynamic tasks, heterogeneous resources, and coupled decisions. In this paper, we investigate the issue of joint service caching and computation offloading in mobile edge-cloud computing networks. Specifically, we formulate the optimization problem as minimizing the long-term average service latency, which is NP-hard. To solve the problem, we conduct in-depth theoretical analyses and decompose it into two sub-problems: service caching processing and computation offloading processing. We are the first to propose a novel hierarchical deep reinforcement learning algorithm to solve the formulated problem, where multiple edge agents and a cloud agent collaboratively determine the caching-action and offloading-action, respectively. The results obtained through trace-driven simulations reveal that the proposed framework outperforms several prevailing algorithms concerning the average service latency across diverse scenarios. In a complex real scenario, our framework achieves an approximately 33% convergence improvement and a remarkable 39% reduction in the average service latency when compared to reinforcement learning-based algorithms.**

***Index Terms*—Computation offloading, hierarchical deep reinforcement learning, mobile edge-cloud computing, service caching.**

Manuscript received 2 August 2023; revised 26 December 2023; accepted 11 January 2024. Date of publication 19 January 2024; date of current version 8 August 2024. This work was supported in part by the National Key R&D Program of China under Grant 2022YFE0125400, in part by the National NSFC under Grants 62372072, 62102053, 62072060, 92067206, 61972222, and 62322601, in part by the Chongqing Research Program of Basic Research and Frontier Technology under Grant cstc2022ycjhbgzxm0058, in part by the Key Research Program of Chongqing Science & Technology Commission under Grant cstc2021jscx-dxwtBX0019, in part by the Science and Technology Plan Project of Chongqing Economic and Information Commission under Grant 2211R49R03, in part by the Haihe Lab of ITAI under Grant 22HHXCJC00002, in part by the Natural Science Foundation of Chongqing, China under Grant CSTB2022NSCQ-MSX1104, in part by the General Program of Chongqing Science & Technology Commission under Grants CSTB2023TIAD-STX0035, CSTB2022TIAD-GPX0017, and CSTB2022TIAD-STX0006, in part by the Key Laboratory of Big Data Intelligent Computing, Chongqing University of Posts and Telecommunications under Grant BDIC-2023-B-003, in part by the Regional Innovation Cooperation Project of Sichuan Province under Grants 2023YFQ0028 and 24QYCX0019, in part by the Regional Science and Technology Innovation Cooperation Project of Chengdu City under Grant 2023-YF11-00023-HZ, in part by the Excellent Youth Foundation of Chongqing under Grant CSTB2023NSCQJQX0025, and in part by Guangdong Pearl River Talent Recruitment Program under Grants 2019ZT08X603 and 2019JC01X235. *(Corresponding author: Xiuhua Li.)*

Chuan Sun and Xiuhua Li are with the School of Big Data & Software Engineering, Chongqing University, Chongqing 400044, China (e-mail: sunchuan0602@gmail.com; lixiuhua1988@gmail.com).

Chenyang Wang and Xiaofei Wang are with the College of Intelligence and Computing, Tianjin University, Tianjin 301600, China (e-mail: chenyangwang@tju.edu.cn; xiaofeiwang@tju.edu.cn).

Qiang He is with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: hqiang@hust.edu.cn).

Victor C. M. Leung is with the College of Computer Science & Software Engineering, Shenzhen University, Shenzhen 518060, China, and also with the Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, BC V6T1Z4, Canada (e-mail: vleung@ieee.org).

Digital Object Identifier 10.1109/TSC.2024.3355937

I. Introduction

EDGE computing has emerged as a promising paradigm for supporting latency-sensitive applications and services by utilizing computation and storage resources closer to end-users, such as edge nodes or base stations [1], [2]. By processing data and running applications closer to the source of data, edge computing can reduce the need for data to be transmitted to the cloud or a centralized data center, thus improving response time and reducing network congestion. Edge computing is being adopted in various domains, such as healthcare, smart cities, and industrial automation [3], [4], [5], to support high-performance computing and low-latency task processing.

However, edge nodes with limited resources have a hard time processing numerous computation-intensive tasks, such as ChatGPT, AlphaGo, and augmented reality [6], [7]. To address this limitation, edge-cloud computing networks have been proposed [8], [9]. By scheduling tasks in a collaborative architecture between the edge and the cloud, and fully utilizing resources across the network, edge-cloud computing networks provide reliable computing power for data processing and task computation, alleviating the tension between computing power and task demand. They can provide distributed, hierarchical, and fine-grained computing, storage, and communication resources for efficient and collaborative task processing, and have become a major goal for future high-performance computing networks [10], [11].

To achieve the above goal, the key to edge-cloud computing networks is how to jointly optimize service caching and computation offloading. *Computation offloading:* lots of existing

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

works have investigated computation offloading in edge-cloud computing networks, which refers to collaboratively offloading tasks between edge nodes and the cloud server to maximize the utilization of edge resources and reduce the execution latency [7], [9], [12], [13]. However, these works ignore service caching and assume that the execution of all tasks requires only computation and storage resources. *Service caching:* several studies have explored the joint optimization of service caching and computation offloading [14], [15], [16], [17], [18], where certain tasks depend on certain services. They address the challenge of deploying services at edge nodes, which consume resources such as memory and storage. To make efficient use of the limited resources, it is crucial to optimize service caching by caching the most suitable services on the edge nodes.

However, the joint service caching and computation offloading problem still faces three significant challenges that must be addressed to achieve efficient system computation performance. These challenges are as follows:

1) *Heterogeneous resources:* Massive edge nodes, distributed worldwide and belonging to different service providers, exhibit highly heterogeneous computing, memory, and storage resources. This heterogeneity poses challenges in effectively integrating and utilizing the resources of edge nodes.
2) *Dynamic tasks:* The distribution of computation tasks at each edge node changes over time due to user mobility, periodicity, and other factors. Additionally, the task load is imbalanced among edge nodes.
3) *Coupled decisions:* Decisions of service caching and computation offloading affect each other and are coupled, making it challenging to find the optimal solution for both decisions. From the time dimension, current service caching decision will affect future caching and offloading decisions.

Given the above challenges, the key open questions are:

1) *Which services to cache at each edge node to best utilize heterogeneous resources?*
2) *How to offload tasks to edge nodes under dynamic environments?*
3) *How to jointly optimize the above coupled decisions?*

Several studies on joint service caching and computation offloading face challenges that are not comprehensively addressed [16], [17], [19]. Recently, deep reinforcement learning (DRL) has emerged as a promising approach for this joint optimization problem [4], [14], [20], [21]. Various central or distributed DRL-based frameworks have been proposed to determine coupled decisions. However, some of these approaches may encounter limitations in effectively decomposing decisions and maintaining personalized models, potentially leading to higher system costs. To address the three questions, in this paper, we propose a joint service caching and computation offloading framework based on hierarchical deep reinforcement learning (HDRL). First, we reasonably decompose joint service caching and computation offloading into service caching processing and computation offloading processing to counter the coupling. Next, we model these processes as Markov decision processes (MDPs) to capture their dynamics. Finally, we propose a novel HDRL algorithm, leveraging the "divide-and-conquer" idea, where multiple agents with deep Q-network (DQN) and Soft Actor-Critic (SAC) can effectively handle the heterogeneity. *To the best of our knowledge, this is the first attempt to optimize joint service caching and computation offloading using the "divide-and-conquer" idea based on HDRL.* Our main contributions are summarized as follows.

- We investigate the joint service caching and computation offloading problem in a mobile edge-cloud computing network. We focus on minimizing the long-term average service latency under multidimensional constraints, which is an integer linear programming problem. We conduct in-depth theoretical analyses and decompose it into two sub-problems: service caching processing and computation offloading processing.
- To effectively handle the two sub-problems, we model them as MDPs. Consequently, we propose a joint service caching and computation offloading framework based on HDRL. Edge agents with a variant of DQN and the central agent with a variant of SAC are responsible for the service caching processing and computation offloading processing, respectively. The collaboration of multi-agents in different hierarchies enables the effective resolution of this complex problem.
- We conduct extensive trace-driven simulations on both synthetic and real datasets. The results demonstrate the superiority of our framework over existing rule-based and reinforcement learning-based algorithms, with notable advancements in convergence and significant reductions in average service latency.

The subsequent sections of this paper are arranged as follows. Section II provides an overview of the related work. Section III presents the system model and formulates the optimization problem. Section IV offers the problem analyses. The joint service caching and computation offloading framework is presented in Section V. The performance of the proposed framework is thoroughly assessed in Section VI. Section VII concludes this paper.

II. Related Work

In this section, we comprehensively review the state-of-the-art literature on the issue of joint service caching and computation offloading.

A. Optimization Within a Single Time Scale

Zhang et al. [22] explored a scenario involving a single edge node and multiple users with multiple tasks. They introduced a semidefinite relaxation-based algorithm designed to address the formulated quadratically constrained quadratic program. In a different perspective, the authors in [29] delved into the issue of computation offloading, particularly in scenarios characterized by low offloading rates. Their investigation encompassed both homogeneous and heterogeneous scenarios, and they proposed two algorithms aimed at maximizing the sum utility. In a similar

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

vein, Chu et al. [23] concentrated on maximizing resource utilization. They presented a Lagrangian-dual-based distributed algorithm tailored to solve the formulated problem. To tackle limited edge node resources, Bi et al. [30] introduced a reduced-complexity alternating minimization technique in a single edge server with a mobile user. Lin et al. [31] proposed an alternating direction method of multipliers-based algorithm for artificial intelligent service caching. Notably, these studies exclusively considered scenarios involving a solitary edge node.

To enhance the collaboration of edge nodes, Li et al. [32] considered a device-to-device assisted edge computing network and proposed a game theory-based suboptimal algorithm. Ma et al. [16] formulated the joint service caching and computation offloading problem as a mixed integer nonlinear programming, which is NP-hard. They proposed a two-layer iterative caching update algorithm to minimize service response time. Considering the interdependence of tasks, Zhao et al. [24] proposed an efficient convex programming-based algorithm to solve the problem of joint service caching and dependent computation offloading. However, these studies focused on scenarios with a singular resource constraint, such as storage or computation.

The authors in [33] investigated the issue with multidimensional constraints, including storage, computation, and communication. They proposed a bi-criteria algorithm to solve the proposed optimization problem with approximation guarantees. Considering the competitive relationships among edge nodes, Zhang et al. [34] focused on social welfare maximization and proposed an auction-based resource allocation mechanism to enhance the effectiveness of allocated resources. Fan et al. [35] delved into resource management for joint service caching and computation offloading, presenting an alternating optimization-based algorithm to minimize energy consumption. In a spatial-random two-tier heterogeneous network, Zhang et al. [25] proposed a stochastic geometry and queuing theory-based algorithm to minimize average delay. Addressing the heterogeneity of task requests, Zhong et al. [15] introduced an efficient cooperative service caching and computation offloading algorithm. Despite these valuable contributions, it's noteworthy that the aforementioned studies primarily focused on optimizing within a single time scale, overlooking the dynamic nature inherent in edge computing networks.

B. Optimization Within Multiple Time Scales

To optimize service utilization within resource constraints for long-term efficiency, Xu et al. [19] proposed an online service caching algorithm for mobile edge computing. Addressing the interplay between service caching and computation offloading in intelligent transportation systems, Xu et al. [28] introduced a decision theory-based algorithm to minimize system latency. Examining challenges in vehicular networks with dynamic topologies and limited caching space, Xue et al. [20] applied a deep reinforcement learning algorithm to address mixed-integer nonlinear programming. However, these studies assumed resource homogeneity. In reality, edge nodes, often belonging to different service providers, exhibit significant heterogeneity in computing, memory, and storage resources.

To address the heterogeneity, the authors in [17] proposed a two-time-scale framework under storage, communication, computation, and budget constraints. They further proposed two approximation algorithms for service caching and computation offloading, respectively. Zhou et al. [27] proposed a gradient descent allocation algorithm and a game theory-based computing strategy to minimize the task cost. Liu et al. [36] proposed two heuristic-based algorithms for service caching and computation offloading to balance the server load. Zhou et al. [18] proposed an alternating optimization-based algorithm by leveraging Lyapunov optimization approach and dependent rounding technique. Fan et al. [26] proposed an iterative algorithm based on multiple numerical techniques to minimize the total task processing delay. Despite these contributions, these approaches lack intelligence and show limited effectiveness in dynamically evolving and complex edge computing environments.

Recently, deep reinforcement learning has found application in the domain of joint service caching and computation offloading [4], [14], [21]. Zhou et al. [4] proposed a deep deterministic policy gradient-based algorithm to determine the strategies of computation offloading, service caching, and resource allocation. They use a central model for all decisions, which is inefficient and takes a long time to converge under large-scale scenarios. To address this problem, Ren et al. [14] designed two proximal policy optimization-based online algorithms to determine where to execute computation tasks and whether to cache the services, respectively. Zhou et al. [21] further proposed an asynchronous advantage actor-critic-based algorithm to minimize the cost, where multiple agents learn asynchronously. However, these agents shared the same model, limiting the ability to maintain personalized models for distributed decisions, such as individual caching decisions for each edge node. Furthermore, existing studies overlook the time consumption associated with service caching and deployment, introducing complexity to the optimization process.

C. Summary

In Table I, we provide a succinct summary of the disparities between our work and the most relevant studies. It is evident that the existing work falls short in adequately addressing three pivotal challenges. In contrast, our work adeptly tackles these challenges in a simultaneous and comprehensive manner.

III. System Model and Problem Formulation

We consider a mobile edge-cloud computing network consisting of multiple heterogeneous edge nodes and a cloud server, where edge nodes are equipped with limited computation, storage, memory, and communication capacities, as shown in Fig. 1. Let $\mathcal{E} = \{1, 2, \ldots, E\}$ denote the set of edge nodes. Edge node $e \in \mathcal{E}$ has total computing resources C_e (measured in CPU cycles per second), total memory resources R_e (measured in GB), and total storage resources B_e (measured in GB). In the considered network, we assume that edge nodes can communicate with each other via the same local area network (LAN) or wired peer-to-peer links [16]. All edge nodes are connected to the remote cloud (denoted by o) via the Internet. The considered

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DISPARITIES BETWEEN OUR WORK AND THE MOST RELEVANT WORK

Work	Dynamic tasks	Heterogeneous resources	Edge node collaboration	Multidimensional constraints	Intelligence
[22], [23]	×	√	×	√	×
[16], [24]	×	√	√	×	×
[15], [25]	×	×	√	√	×
[26], [27]	√	√	√	√	×
[19], [28]	√	×	√	×	×
Our work	√	√	√	√	√

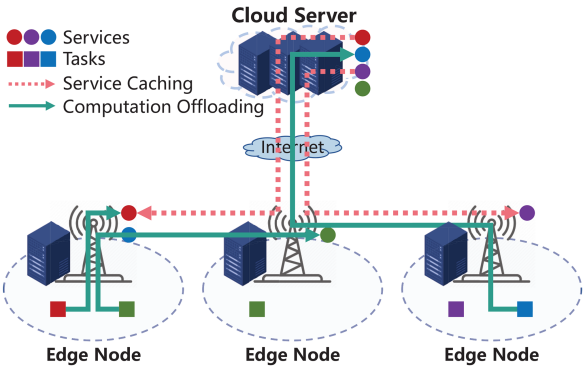


Fig. 1. Architecture of a mobile edge-cloud computing network.

TABLE II
SUMMARY OF PRIMARY NOTATIONS

Parameters	Notations
$\mathcal{E}, \mathcal{S}, \mathcal{T}$	Set of edge nodes, set of services, and set of time slots
$\mathcal{X}, \mathcal{Y}$	Caching decision and offloading decision
C_e, R_e, B_e	Computing, memory, and storage resources of node e
d_s, r_s, b_s	Data, memory, and storage sizes of service s
$\widehat{d}_s$, c_s	Data size and number of CPU cycles of tasks s
$v_s^e(t)$	Number of tasks s at edge node e
W, $\widehat{W}$	Average transmission rates of Internet and of LAN
C_o	Computing resource of the cloud server

network is stable. In other words, there will be no downtime for nodes and connections. Table II shows some primary notations.

We focus on joint service caching and computation offloading in the mobile edge-cloud computing network. Each edge node covers a specific area, where mobile users can offload their various computation tasks to the edge node. Specifically, the execution of computation tasks requires that the edge node deploys specific services [26]. We first cache fractional services from the cloud server to edge nodes (*service caching*). We then offload tasks to match corresponding services due to resource-constrained edge nodes (*computation offloading*). For instance, in Fig. 1, the tasks (red) can be executed locally due to the cached service (red), but the tasks (green) should be offloaded to the adjacent edge node with the corresponding service (green), or the tasks (blue) are offloaded to the cloud server.

Let $\mathcal{S} = \{1, 2, \ldots, S\}$ denote the set of service types at the cloud server. These service types, such as mobile game, virtual reality, and neural network training, exhibit distinct characteristics. Let d_s denote the data size of service s, which will affect the communication latency. Additionally, service s will occupy a certain amount of memory size r_s and storage size b_s when it is successfully deployed at an edge node. For brevity, we refer to tasks and service of type s as tasks s and service s, respectively.

To account for the dynamic nature of the network environment, we consider a time-slotted mode of operation for the system, where the time slots are denoted by $\mathcal{T} = \{1, 2, \ldots, T\}$. We assume that the arrival of tasks s in the coverage area of edge node e follows a Poisson distribution [16], [37]. Let $v_s^e(t)$ denote the number of tasks $s \in \mathcal{S}$ at edge node $e \in \mathcal{E}$ at the beginning of time slot $t \in \mathcal{T}$.[1]

A. Service Caching Model

In this subsection, we introduce how to cache services in detail. As all edge nodes can cache fractional services to support computation tasks, we consider each edge node independently decides whether to cache related services.

Definition 1: The *caching decision* is defined as integer variables $\mathcal{X} = \{x_{e,s}(t)\}_{e\in\mathcal{E}, s\in\mathcal{S}, t\in\mathcal{T}}$, where $x_{e,s}(t) \in \{0, 1\}$ indicates whether service s is cached at edge node e ($x_{e,s}(t) = 1$) or not ($x_{e,s}(t) = 0$) at the beginning of time slot t.

We consider that each edge node caches at most one service in each time slot [39], given by

$$\text{C1: } \sum_{s\in\mathcal{S}} x_{e,s}(t) \le 1, \forall e \in \mathcal{E}, \forall t \in \mathcal{T}. \tag{1}$$

We denote the cached service information of edge node e at the beginning of time slot t as $(u_s^e(t))_{s\in\mathcal{S}}$. Specifically, if $x_{e,s}(t) = 1$, we set $u_s^e(t) = 1$; otherwise, $u_s^e(t) = 0$. As the edge nodes are resource-limited, they cannot cache all services and have the following constraints

$$\text{C2: } \sum_{s\in\mathcal{S}} u_s^e(t) r_s \le R_e, \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \tag{2}$$

$$\text{C3: } \sum_{s\in\mathcal{S}} u_s^e(t) b_s \le B_e, \forall e \in \mathcal{E}, \forall t \in \mathcal{T}. \tag{3}$$

As previously mentioned, the execution of computation tasks depends on the corresponding services, which can be thought of as programs. When a service has been cached and installed at a certain edge node, the entity of this service will be destroyed. Therefore, services can only be cached from the cloud server and cannot be cached from other edge nodes by the edge-edge cooperation. Once the caching decision is determined in *Definition 1*, each edge node will cache the service from the cloud server through the Internet. Besides, the data size of the

[1] Although edge nodes have limited storage capacity (measured in GB), computation tasks are usually computation-intensive and have limited data size (measured in Mbit). Hence, each edge node is assumed to receive all tasks offloaded from serving mobile users [14], [38].

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

service only affects communication latency (as explained in Sect. III-C) and does not occupy the storage resource of edge nodes. Particularly, when edge node e caches enough services after several time slots, it replaces old services with a first-in first-out (FIFO) manner. The cached service information will change, i.e., $u_s^e(t) = 0$.

B. Computation Offloading Model

In this subsection, we will provide a detailed explanation of how to offload tasks. When an edge node does not have the services required to execute a particular task, the edge node must offload the task either to adjacent edge nodes with the corresponding services or to the cloud server.

Definition 2: The *offloading decision* is defined as integer variables $\mathcal{Y} = \{y_{e,s}(t)\}_{e\in\mathcal{E}\cup\{o\},s\in\mathcal{S},t\in\mathcal{T}}$, where $y_{e,s}(t) \in \{0,1\}$ indicates whether tasks s are executed at edge node e or the cloud server o at the beginning of time slot t.

We consider that tasks s are only executed at one place [26], which is expressed as follows

$$\text{C4:} \sum_{e\in\mathcal{E}\cup\{o\}} y_{e,s}(t) = 1, \forall s \in \mathcal{S}, \forall t \in \mathcal{T}. \tag{4}$$

Similarly, computation offloading should meet the following constraints

$$\text{C5:} \sum_{s\in\mathcal{S}} y_{e,s}(t)\widehat{d}_s V_s(t) \le B_e, \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \tag{5}$$

$$\text{C6:} \sum_{s\in\mathcal{S}} [y_{e,s}(t)\widehat{d}_s V_s(t) + u_s^e(t) b_s] \le B_e, \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \tag{6}$$

where $V_s(t) = \sum_{e\in\mathcal{E}} v_s^e(t)$ is the overall number of tasks s. The constraint C6 is derived by C3 and C5 because both tasks and services consume the storage resource of each edge node. Once the offloading decision is determined in *Definition 2*, tasks s at all edge nodes should be offloaded to the target execution location (edge node e via LAN or the cloud server via the Internet).

C. Communication and Computation Model

In this subsection, we introduce the service latency of service caching and computation offloading, which mainly consist of some communication and computation latency.

1) Communication latency: During the process of service caching, service s cached from the cloud server to the edge nodes will generate some communication latency, given by $l_{e,s}^{tra}(t) = d_s/W, s \in \mathcal{S}, e \in \mathcal{E}$, where W is the average transmission rate of Internet. Similarly, during the process of computation offloading, tasks s offloaded from edge node e to adjacent edge node i or the cloud server also generate some communication latency, given by

$$l_{e,i,s}^{tra}(t) = \begin{cases} \frac{\widehat{d}_s}{\widehat{W}}, & \text{if } i \in \mathcal{E} \setminus e \\ 0, & \text{if } i = e \\ \frac{\widehat{d}_s}{\widehat{W}}, & \text{if } i = o \end{cases}$$
$$\forall e \in \mathcal{E}, \forall i \in \mathcal{E} \cup \{o\}, \forall s \in \mathcal{S}, \forall t \in \mathcal{T}, \tag{7}$$

where $\widehat{d}_s$ is the data size of tasks s; $\widehat{W}$ is the average transmission rate of LAN. Particularly, $i \in \mathcal{E} \setminus e$ represents that i belongs to the set without e because intra-node offloading does not have communication latency.

2) Computation latency: When service caching and computation offloading are finished, the execution of tasks s will generate computation latency, given by $l_{e,s}^{com}(t) = c_s/C_e, \forall s \in \mathcal{S}, \forall e \in \mathcal{E} \cup \{o\}$, where c_s is the number of CPU cycles. Particularly, C_o is the average computing resource of the cloud server.

D. Problem Formulation

We focus on jointly optimizing service caching and computation offloading policies to minimize the long-term average service latency. The total service latency in time slot t includes communication and computation latency, given by

$$\begin{aligned} L(t) &= \underbrace{\sum_{s\in\mathcal{S}}\sum_{e\in\mathcal{E}} x_{e,s}(t) l_{e,s}^{tra}(t)}_{\text{part I}} + \underbrace{\sum_{s\in\mathcal{S}}\sum_{e\in\mathcal{E}\cup\{o\}} y_{e,s}(t) l_{e,s}^{com}(t) V_s(t)}_{\text{part II}} \\ &\quad + \underbrace{\sum_{s\in\mathcal{S}}\sum_{e\in\mathcal{E}\cup\{o\}} y_{e,s}(t) \sum_{i\in\mathcal{E}} l_{i,e,s}^{tra}(t) v_s^i(t)}_{\text{part III}} \\ &= \sum_{s\in\mathcal{S}} \left[\sum_{e\in\mathcal{E}} x_{e,s}(t) l_{e,s}^{tra}(t) + \sum_{e\in\mathcal{E}\cup\{o\}} y_{e,s}(t) \right. \\ &\quad \left. \times \left[\sum_{i\in\mathcal{E}} l_{i,e,s}^{tra}(t) v_s^i(t) + l_{e,s}^{com}(t) V_s(t) \right] \right], \forall t \in \mathcal{T}, \end{aligned} \tag{8}$$

where part I is the communication latency that service s is cached at edge node e; part II is the computation latency that tasks s are executed at edge node e or the cloud server; part III is the communication latency that tasks s at all edge nodes are offloaded to edge node e or the cloud server.

Finally, the optimization problem is as follows

$$\min_{\mathcal{X},\mathcal{Y}} \lim_{T\to\infty} \frac{1}{T} \sum_{t\in\mathcal{T}} L(t)$$

s.t. C1-C6,

$$x_{e,s}(t) \in \{0,1\}, \forall s \in \mathcal{S}, \forall e \in \mathcal{E}, \forall t \in \mathcal{T},$$
$$y_{e,s}(t) \in \{0,1\}, \forall s \in \mathcal{S}, \forall e \in \mathcal{E} \cup \{o\}, \forall t \in \mathcal{T}, \tag{9}$$

where constraints C1-C6 have been explained before.

IV. Problem Analysis and Decomposition

The optimization problem can be classified as a classic example of Integer Linear Programming (ILP), where all variables must be integers. This problem is widely recognized as NP-hard [33], primarily due to the inclusion of multiple packing constraints (inequalities C2, C3, C5, and C6) that generalize the knapsack problem. The inherent complexity of ILP makes finding an exact optimal solution computationally demanding and often impractical. Furthermore, the spatial and temporal interdependencies between decisions for service caching and

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

computation offloading further increase the complexity of solving this ILP. Moreover, finding an optimal solution requires exhaustively exploring all possible combinations of integer values for decision variables, which becomes computationally intensive as the number of variables and problem size grow. Therefore, it is crucial to analyze the complexity of the formulated problem. In particular, we consider some special cases to gain insights into the problem's complexity and reasonably decompose the original problem.

A. Case 1: Non-Cooperation Among Edge Nodes

To reduce complexity, we initially consider the scenario where there is no cooperation among the edge nodes. Tasks cannot be offloaded among the edge nodes. In this case, the execution of computation tasks is affected, and each edge node has the option to either perform the tasks locally or offload them to the cloud server. Consequently, the offloading decisions can be rewritten as $\mathcal{Y}^1 = \{y^1_{e,s}(t)\}_{e\in\mathcal{E},s\in\mathcal{S},t\in\mathcal{T}}$, where $y^1_{e,s}(t) \in \{0,1\}$ indicates whether tasks s at edge node e are executed locally ($y^1_{e,s}(t)=0$) or offloaded to the cloud server ($y^1_{e,s}(t)=1$). To this end, each edge node can make its own caching and offloading decisions independently. We can then simplify the optimization problem in (9) to a single edge node

$$\min \lim_{T\to\infty} \frac{1}{T} \sum_{t\in\mathcal{T}} \sum_{s\in\mathcal{S}} \Bigg[x_{e,s}(t) l^{tra}_{e,s}(t) + y^1_{e,s}(t) l^{tra}_{e,o,s}(t) + (1-y^1_{e,s}(t)) \frac{c_s}{C_e} + y^1_{e,s}(t) \frac{c_s}{C_o} \Bigg]$$

s.t. C1-C3,

$$\sum_{s\in\mathcal{S}} y^1_{e,s}(t) \widehat{d}_s V_s(t) \le B_e, \forall e \in \mathcal{E}, \forall t \in \mathcal{T},$$

$$x_{e,s}(t), y^1_{e,s}(t) \in \{0,1\}, \forall s \in \mathcal{S}, \forall e \in \mathcal{E}, \forall t \in \mathcal{T}. \quad (10)$$

In this particular case, where each edge node operates independently, our objective is to minimize the service latency of each edge node, leading to the overall minimization of the global service latency. Focusing on the optimization problem presented in (10), we will analyze it starting from time slot 1, where edge node e does not cache any services. Based on the service type configuration outlined in [16], [18], [26], we consider the number of service types as $S=8$. When $t=1$, the maximum possible combinations in (10) are $2^S \times S = 2048$. However, since the edge node has already cached relevant services, the number of possible combinations is significantly lower than 2048. In this case, we can employ greedy algorithms to efficiently determine the optimal solution for each time slot.

B. Case 2: Optimization Within One Time Slot

We then consider the optimization within one time slot, which can eliminate temporal coupling. The problem in (9) within one time slot can be reformulated as

$$\min \sum_{s\in\mathcal{S}} \Bigg[\sum_{e\in\mathcal{E}} x_{e,s}(t) l^{tra}_{e,s}(t) + \sum_{e\in\mathcal{E}\cup\{o\}} y_{e,s}(t) \times \Bigg[\sum_{i\in\mathcal{E}} l^{tra}_{i,e,s}(t) v^i_s(t) + l^{com}_{e,s}(t) V_s(t) \Bigg] \Bigg]$$

s.t. Constrains in (9). (11)

By relaxing the offloading decision variable $y_{e,s}(t)$ to be fractional, where $y_{e,s}(t) \in [0,1]$ represents the ratio of tasks of type s executed at edge node e in time slot t, the optimization problem can be transformed into the joint service caching and workload scheduling problem discussed in [16]. In their work, the authors extensively analyzed this problem, which consists of two interconnected subproblems. The service caching policies will affect how to schedule workload. The results of workload scheduling both reflect the performance of the workload scheduling policies and the service caching policies. To address this problem, a two-layer iterative caching update algorithm is proposed, where the outer layer updates the service caching policies and the inner layer determines the workload scheduling policies.

C. Problem Decomposition

Based on the aforementioned analysis, we can attempt to decompose the problem presented in (9) into several subproblems. To facilitate the analysis, let's consider a homogeneous environment, where $C_1 = \ldots = C_E$, $R_1 = \ldots = R_E$, and $B_1 = \ldots = B_E$, starting from time slot $t=1$. At the beginning of time slot $t=1$, various computation tasks are present at the edge nodes. Since task execution relies on the corresponding services, our initial step is to determine the services that will be cached from the cloud server (*caching decision*) in order to meet a larger number of tasks. The following lemma provides insight into this process.

Lemma 1: The caching decisions $\{x_{e,s}(1)\}_{e\in\mathcal{E},s\in\mathcal{S}}$ can be determined by the current task load $\{v^e_s(1)\}_{e\in\mathcal{E},s\in\mathcal{S}}$.

Proof: We assume that tasks $s^* \in \mathcal{S}$ have the maximum size compared to other task types at edge node $e \in \mathcal{E}$, which can be expressed as $\widehat{d}_{s^*} v^e_{s^*}(1) = \max\{\widehat{d}_s v^e_s(1) | \forall s \in \mathcal{S}\}$. In the beginning of time slot 1, all edge nodes do not cache any services. Besides, each edge node can cache at most one service in each time slot. If edge node e caches service s^*, tasks s^* will be executed locally, its service latency only includes computation latency, i.e., $l^{loc}_{e,s^*}(1) = c_{s^*} v^e_{s^*}(1)/C_e$. Instead, tasks s^* at edge node e should be offloaded to edge node $i \in \mathcal{E} \setminus e$ or the cloud server o, its service latency includes communication latency and computation latency, i.e., $l^{off}_{e,s^*}(1) = (l^{tra}_{e,i,s^*}(1) + l^{com}_{i,s^*}(1)) v^e_{s^*}(1)$. As the homogeneous environment, all edge nodes have the same computation capability, i.e., $C_e = C_i, e \neq i$. Apparently, $l^{off}_{e,s^*}(1) > l^{loc}_{e,s^*}(1)$.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

If edge node e caches service s^* at the beginning of time slot 1, the service latency of the edge node is

$$L_e^{s^*}(1) = l_{e,s^*}^{loc}(1) + \sum_{s\in\mathcal{S}\setminus s^*} \left(l_{e,i,s}^{tra}(1) + l_{i,s}^{com}(1)\right) v_s^e(1), \quad \forall e \in \mathcal{E}, i \in \{o\} \cup \mathcal{E} \setminus e.$$

Instead, if edge node e caches service $\neg s^*, \neg s^* \neq s^*$, the service latency of the edge node is

$$L_e^{\neg s^*}(1) = \sum_{s\in\mathcal{S}\setminus\{\neg s^*, s^*\}} \left(l_{e,i,s}^{tra}(1) + l_{i,s}^{com}(1)\right) v_s^e(1) + l_{e,s^*}^{off}(1) + l_{e,\neg s^*}^{loc}(1), \forall e \in \mathcal{E}, i \in \{o\} \cup \mathcal{E} \setminus e.$$

To compare the values of $L_e^{s^*}(1)$ and $L_e^{\neg s^*}(1)$, we assume that other tasks have the same offloading decisions apart from s^* and $\neg s^*$. Besides, tasks s^* and $\neg s^*$ are executed at edge nodes. Therefore, we have

$$\begin{aligned} L_e^{\neg s^*}(1) - L_e^{s^*}(1) &= \frac{c_{\neg s^*} v_{\neg s^*}^e(1)}{C_e} + \left(\frac{\widehat{d}_{s^*}}{\widehat{W}} + \frac{c_{s^*}}{C_i}\right) v_{s^*}^e(1) \\ &\quad - \frac{c_{s^*} v_{s^*}^e(1)}{C_e} - \left(\frac{\widehat{d}_{\neg s^*}}{\widehat{W}} + \frac{c_{\neg s^*}}{C_i}\right) v_{\neg s^*}^e(1) \\ &= \frac{\widehat{d}_{s^*} v_{s^*}^e(1)}{\widehat{W}} - \frac{\widehat{d}_{\neg s^*} v_{\neg s^*}^e(1)}{\widehat{W}} \geq 0. \end{aligned}$$

Therefore, we can reduce the service latency by caching the service that the related tasks have the maximum size. Similarly, we can generalize to all edge nodes and more time slots. This completes the proof.

Lemma 2: The offloading decisions $\{y_{e,s}(1)\}_{e\in\mathcal{E},s\in\mathcal{S}}$ should be determined after the caching decisions $\{x_{e,s}(1)\}_{e\in\mathcal{E},s\in\mathcal{S}}$ are determined.

Proof: According to Lemma 1, we can determine how to cache services at edge nodes. For the tasks with the maximum size at edge node e, we can determine these tasks executed locally to reduce the service latency. However, we cannot determine how to offload other tasks due to no services at edge nodes. Therefore, optimizing the offloading decisions before the caching decisions are determined would be meaningless. After the caching decisions are determined, $(u_s^e(t))_{s\in\mathcal{S}}$ will be updated. Based on the cached service information and local task information, we can further determine the offloading decisions. Moreover, we can infer that the caching decision will affect the offloading decision, which highlights the importance of prioritizing the caching decision. This completes the proof.

Theorem 1: The problem in (9) in each time slot can be decomposed into the *service caching processing* in (13) and the *computation offloading processing* in (14).

Proof: According to Lemmas 1, we should first determine the caching decisions based on the current task load. According to Lemma 2, we then determine the offloading decisions based on the determined caching decisions. We further rewrite the service latency in (8)

$$L(t) = \underbrace{\sum_{s\in\mathcal{S}} \sum_{e\in\mathcal{E}\cup\{o\}} y_{e,s}(t)[l_{e,s}^{com}(t)V_s(t) + \sum_{i\in\mathcal{E}} l_{i,e,s}^{tra}(t) v_s^i(t)]}_{\text{computation offloading}} + \underbrace{\sum_{s\in\mathcal{S}} \sum_{e\in\mathcal{E}} x_{e,s}(t) l_{e,s}^{tra}(t)}_{\text{service caching}}, \forall t \in \mathcal{T}. \quad (12)$$

We observe that the caching decision $x_{e,s}(t)$ mainly affects the communication latency of services and the offloading decision $y_{e,s}(t)$. $y_{e,s}(t)$ mainly affects the communication and computation latency of tasks. Many existing related work in [17], [40] also solves the joint service caching and request scheduling problem as by separately solving service caching and request scheduling. Therefore, we can solve the problem in (9) by solving service caching and computation offloading sequentially. We first fix $\{y_{e,s}^*(t)\}_{e\in\mathcal{E},s\in\mathcal{S}}$ to optimize $\{x_{e,s}(t)\}_{e\in\mathcal{E},s\in\mathcal{S}}$ by the following service caching processing problem

$$\min_{x_{e,s}(t)} L(y_{e,s}^*(t), t) \quad \text{s.t. C1-C3}, x_{e,s}(t) \in \{0,1\}, \forall s \in \mathcal{S}, \forall e \in \mathcal{E}. \quad (13)$$

After determining $\{x_{e,s}^*(t)\}_{e\in\mathcal{E},s\in\mathcal{S}}$, we further optimize $\{y_{e,s}(t)\}_{e\in\mathcal{E},s\in\mathcal{S}}$ by the following computation offloading processing problem

$$\min_{y_{e,s}(t)} L(x_{e,s}^*(t), t) \quad \text{s.t. C4-C6}, y_{e,s}(t) \in \{0,1\}, \forall s \in \mathcal{S}, \forall e \in \mathcal{E} \cup \{o\}. \quad (14)$$

We can sequentially solve the *service caching processing* and the *computation offloading processing* to get an approximate optimal solution of the problem in (9). This completes the proof.

V. Joint Service Caching and Computation Offloading Framework

In this section, we present a joint service caching and computation offloading framework (JSC2O) based on HDRL.

A. Why Hierarchical Deep Reinforcement Learning?

The problem in (9) poses significant challenges even when using traditional approaches. Despite being divided into two subproblems, solving the problem using conventional methods remains highly complex. Particularly, employing exhaustive search algorithms to find the optimal $x_{e,s}^*(t)$ and $y_{e,s}^*(t)$ is impractical due to the exponential search space. For example, let's consider a simple scenario with $E = 5$ and $S = 8$. This implies that applying the exhaustive approach would require searching through $(S+1)^E = 59,049$ possible combinations for $x_{e,s}(t)$ and $2^{(E+1)S} \approx 2.8 \times 10^{14}$ possible combinations for $y_{e,s}(t)$. This is quite difficult and impractical. Besides, it is impossible to transform the problem into a geometric programing due to constraints C1 and C4 with the nonlinearity of the objective [41].

Reinforcement learning interacts continuously with complex and dynamic environments to obtain feedback information in

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

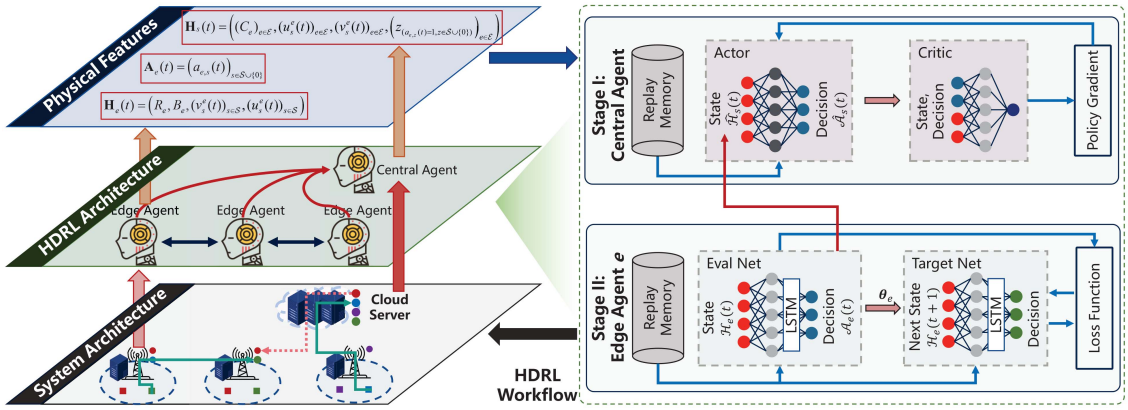


Fig. 2. Framework of JSC2O.

the form of rewards and punishments, gradually approaching the optimal solution. Specifically, reinforcement learning also has the ability to capture the dynamic characteristics of the environment, making it an important theoretical technology for optimizing decision-making in complex and dynamic mobile edge-cloud computing networks. Meanwhile, determining the caching decision mainly depends on local task information, but determining the offloading decision needs global service caching and task information. To this end, we leverage the idea of "divide-and-conquer" and propose a JSC2O framework based on HDRL.

The proposed JSC2O framework can be regarded as a multi-layer architecture: system architecture, HDRL architecture, and physical features, as shown in Fig. 2. The HDRL architecture is composed of several edge agents and one central agent, which respectively correspond to the edge nodes and the cloud server in the system architecture. In each time slot, HDRL has two stages to solve the service caching processing and computation offloading processing. The HDRL architecture can extract some features from the system architecture to form the physical features.

Specifically, the service caching processing and computation offloading processing are modeled as MDPs. The edge agent at edge node $e \in \mathcal{E}$ is responsible for learning an optimal policy π_e^* to independently determine the caching decision. Simultaneously, the central agent at the cloud server is responsible for learning an optimal policy π_o^* to determine the offloading decision. In accordance with Theorem 1, the service caching processing and computation offloading processing are determined sequentially. Consequently, the JSC2O framework consists of two stages. In Stage I, all edge agents independently generate caching-actions, uploading these actions to the cloud server. In Stage II, the cloud agent generates the offloading-actions, downloading these actions to all edge nodes. All agents cooperate with each other by the same reward function $\Re(t)$ to minimize the long-term average service latency. Therefore, we define the reward function as the negative long-term average service latency, i.e., $\Re(t) = -L(t)$. The fundamental components of these agent models are elaborated upon in the subsequent sections.

B. Stage I: Edge Agents for Service Caching Processing

The service caching MDP at edge node e is described as the tuple $\langle \mathcal{H}_e, \mathcal{A}_e, \mathcal{R} \rangle$, where $\mathcal{H}_e = \{\mathbf{H}_e(t)\}_{t\in\mathcal{T}}$, $\mathcal{A}_e = \{\mathbf{A}_e(t)\}_{t\in\mathcal{T}}$, and $\mathcal{R} = \{\Re(t)\}_{t\in\mathcal{T}}$ are caching-state, caching-action, and reward. The detailed definitions are as follows.

i) Caching-state: The capacity of edge nodes will affect the edge agents from making caching decisions. The edge agent needs to know current computation task information and cached service information of edge node e in time slot t. Computation task information is $(v_s^e(t))_{s\in\mathcal{S}}$ (the number of tasks s). As mentioned above, we denote $(u_s^e(t))_{s\in\mathcal{S}}$ as the cached service information, where $u_s^e(t) \in \{0,1\}$ indicates whether service s has been cached at edge node e ($u_s^e(t) = 1$) or not ($u_s^e(t) = 0$). Finally, the caching-state $\mathcal{H}_e(t)$ is denoted as the combination of (R_e, B_e), $(v_s^e(t))_{s\in\mathcal{S}}$ and $(u_s^e(t))_{s\in\mathcal{S}}$, given as

$$\mathbf{H}_e(t) = \left(R_e, B_e, (v_s^e(t))_{s\in\mathcal{S}}, (u_s^e(t))_{s\in\mathcal{S}}\right), \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \tag{15}$$

which is a $(2S+2)$-dimensional tuple.

ii) Caching-action: The edge agent makes an action on which service to cache. Let $\mathbf{A}_e(t)$ denote the caching-action, given by

$$\mathbf{A}_e(t) = \left(a_{e,s}(t)\right)_{s\in\mathcal{S}\cup\{0\}}, \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \tag{16}$$

where $a_{e,s}(t) \in \{0,1\}$ indicates whether service s is cached at edge node e ($a_{e,s}(t) = 1$) or not ($a_{e,s}(t) = 0$). Particularly, $a_{e,0}(t)$ represents that edge node e does not cache any services.

The caching-action $a_{e,s}(t)$ should be converted into the caching decision $x_{e,s}(t)$. As edge node e has cached some services, the agent may generate some unbecoming caching-actions. If $a_{e,s}(t) = 1$, setting $x_{e,s}(t) = 1$ should meet $u_s^e(t) = 0$, otherwise $x_{e,s}(t) = 0$, given by

$$x_{e,s}(t) = \begin{cases} 1, & \text{if } a_{e,s}(t) = 1, u_s^e(t) = 0 \\ 0, & \text{otherwise} \end{cases}$$
$$\forall e \in \mathcal{E}, \forall s \in \mathcal{S}, \forall t \in \mathcal{T}. \tag{17}$$

In particular, when $a_{e,0}(t) = 1$, $x_{e,s}(t) = 0$ for all $s \in \mathcal{S}$ and satisfies constraint C1. To satisfy constraints C2 and C3, edge node e should pop the first service in the queue by using the FIFO manner as mentioned above.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

As the edge agents at the edge nodes receive simple information to generate discrete actions, we use a simple and efficient DQN, which is often used in dynamic systems without a priori knowledge. DQN aims to learn an optimal policy $\pi_e^* : \mathcal{H}_e \rightarrow \mathcal{A}_e$ for generating actions based on states. We use the function approximation way (i.e., deep neural networks (DNNs)) to fit the policy. Under a given policy π_e, the true Q-value is

$$Q_{\pi_e}(\mathcal{H}_e, \mathcal{A}_e) \equiv \mathbb{E}\left[\mathfrak{R}(1) + \gamma\mathfrak{R}(2) + \ldots\right], \forall e \in \mathcal{E}, \quad (18)$$

where $\gamma \in [0, 1]$ is a discount factor balancing the importance of immediate and future rewards. Denote $\boldsymbol{\theta}_e$ as the parameters of DNN. We can update the parameters $\boldsymbol{\theta}_e$ by applying the caching-action $\mathbf{A}_e(t)$ into the current caching-state $\mathbf{H}_e(t)$ to get the immediate reward $\mathfrak{R}(t+1)$ and the next caching-state $\mathbf{H}_e(t+1)$, which is written as

$$\boldsymbol{\theta}_e(t+1) = \boldsymbol{\theta}_e(t) + \alpha\left(K_e^{Q_{\pi_e}}(t) - Q_{\pi_e}\left(\mathbf{H}_e(t), \mathbf{A}_e(t); \boldsymbol{\theta}_e(t)\right)\right) \nabla_{\boldsymbol{\theta}_e(t)} Q_{\pi_e}\left(\mathbf{H}_e(t), \mathbf{A}_e(t); \boldsymbol{\theta}_e(t)\right), \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \quad (19)$$

where α is learning rate; $\nabla_{\boldsymbol{\theta}_e(t)} Q_{\pi_e}(\cdot)$ is the gradient; $K_e^{Q_{\pi_e}}(t)$ is defined as

$$K_e^{Q_{\pi_e}}(t) \equiv \mathfrak{R}(t+1) + \gamma \max_{\mathbf{A}_e(t)} Q_{\pi_e}\left(\mathbf{H}_e(t), \mathbf{A}_e(t); \boldsymbol{\theta}_e(t)\right), \quad \forall e \in \mathcal{E}, \forall t \in \mathcal{T}. \quad (20)$$

To improve the exploration ability of DQN, some caching-actions are selected randomly, given by

$$\mathbf{A}_e(t) = \begin{cases} \text{a random action}, & \text{w.p. } 1-\epsilon \\ \arg\max_{\mathcal{A}_e} Q_{\pi_e}\left(\mathbf{H}_e(t), \mathcal{A}_e; \boldsymbol{\theta}_e(t)\right), & \text{w.p. } \epsilon \end{cases} \quad \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \quad (21)$$

where 'w.p.' is the short-form for "with probability", and $1-\epsilon$ is the probability of random exploration;

The resource consumption of edge agents is less than the resource required by computation-intensive tasks, we consider that training and inference of DQN are at edge nodes. DQN training at edge node e is given in Algorithm V-B. We refer the tuple $\langle \mathbf{H}_e(t), \mathbf{A}_e(t), \mathbf{H}_e(t+1), \mathfrak{R}(t)\rangle$ to $\mathcal{M}_e$ a experience. Each edge node maintains a replay memory $\mathcal{M}_e$ to store experience. Meanwhile, edge node e includes an evaluation DNN and a target DNN, which are used for caching-action selection and characterizing a target Q-value, respectively.

Algorithm 1: DQN Training at Edge Node e.

Input: $\mathcal{T}, \alpha, \gamma, \epsilon$, service and task information, and Episodes.
Output: π_e^*.

```
Initialize the DQN model;
Initialize replay memory M_e;
while episode in Episodes do
    Reset the mobile edge-cloud computing network;
    for time slot t ∈ T do
        Observe the current caching-state H_e(t);
        Select a caching-action A_e(t) by (21);
        Convert A_e(t) by (17);
        Execute the caching decision in the network;
        Observe the next caching-state H_e(t+1);
        Get the reward ℜ(t);
        Construct and save
          ⟨H_e(t), A_e(t), H_e(t+1), ℜ(t)⟩ to M_e;
        Randomly select a mini-batch from M_e;
        Update the parameters θ_e by (19);
    end
end
Save this model;
```

C. Stage II: Central Agent for Computation Offloading Processing

The computation offloading MDP at the cloud server is described as the tuple $\langle \widehat{\mathcal{H}}, \widehat{\mathcal{A}}, \mathcal{R}\rangle$, where $\widehat{\mathcal{H}} = \{\widehat{\mathbf{H}}_s(t)\}_{s\in\mathcal{S}, t\in\mathcal{T}}$ and $\widehat{\mathcal{A}} = \{\widehat{\mathbf{A}}_s(t)\}_{s\in\mathcal{S}, t\in\mathcal{T}}$ are offloading-state and offloading-action. The detailed definitions are given follows.

i) Offloading-state: We consider the central agent separately makes the offloading-action for each service. The offloading-state includes four parts: computing resource of edge nodes, cached service information about service s, computation task information about tasks s, and caching-action information of all edge agents. Let $(z_{(a_{ez}(t)=1, z\in\mathcal{S}\cup\{0\})})_{e\in\mathcal{E}}$ denote the caching-action information, where $z_{(a_{ez}(t)=1, z\in\mathcal{S}\cup\{0\})}$ represents getting index z when $a_{ez}(t) = 1$. Let $\widehat{\mathbf{H}}_s(t)$ denote the offloading-state, given by

$$\widehat{\mathbf{H}}_s(t) = ((C_e)_{e\in\mathcal{E}}, (u_s^e(t))_{e\in\mathcal{E}}, (v_s^e(t))_{e\in\mathcal{E}}, \left(z_{(a_{e,z}(t)=1, z\in\mathcal{S}\cup\{0\})}\right)_{e\in\mathcal{E}}), \forall s \in \mathcal{S}, \forall t \in \mathcal{T}, \quad (22)$$

which is a $4E$-dimensional tuple.

ii) Offloading-action: The central agent generates an action on which edge node to execute tasks s. Let $\widehat{\mathbf{A}}_s(t)$ denote the caching-action, given by

$$\widehat{\mathbf{A}}_s(t) = \left(\widehat{a}_{e,s}(t)\right)_{e\in\mathcal{E}\cup\{o\}}, \forall s \in \mathcal{S}, \forall t \in \mathcal{T}, \quad (23)$$

where $\widehat{a}_{e,s}(t) \in \{0, 1\}$ indicates whether tasks s are executed at edge node e ($\widehat{a}_{e,s}(t) = 1$) or not ($\widehat{a}_{e,s}(t) = 0$). Particularly, $a_{o,s}(t)$ represents that tasks s are executed at the cloud server.

Similarly, we converted the offloading-action $\widehat{a}_{e,s}(t)$ into the offloading decision $y_{e,s}(t)$. Meanwhile, the place should cache related services to support task execution. Besides, the offloading decision should meet constrains C4-C6. The conversion rule is expressed as

$$y_{e,s}(t) = \begin{cases} 1, & \text{if } \widehat{a}_{e,s}(t) = 1, u_s^e(t) = 1, C4-C6 \\ 0, & \text{otherwise} \end{cases} \quad \forall e \in \mathcal{E}\cup\{o\}, \forall s \in \mathcal{S}, \forall t \in \mathcal{T}. \quad (24)$$

As the central agent at the cloud server needs gathers global complex information to make discrete actions, we propose a discrete SAC (DSAC) to address this issue. SAC with a maximum entropy objective can fully explore the state space $\widehat{\mathcal{H}}$ to avoid he policy from falling into local optimum. SAC consists of two neural networks: an actor network and a critic network. The actor is used to generate actions, while the critic network evaluates the

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: DSAC Training at the Cloud Server.

Input: $\mathcal{T}, \mathcal{S}$, service and task information, caching-action information of DQNs, and Episodes.
Output: π_o^*.

Initialize the DSAC model;
Initialize replay memory $\mathcal{M}_o$;
while *episode in Episodes* **do**
  Reset the mobile edge-cloud computing network;
  for *time slot* $t \in \mathcal{T}$ **do**
    for *edge node* $e \in \mathcal{E}$ **do**
      Get caching-action information by Algorithm 1;
    end
    for *service* $s \in \mathcal{S}$ **do**
      Observe the current offloading-state $\widehat{\mathbf{H}}_s(t)$;
      Select an offloading-action $\widehat{\mathbf{A}}_s(t) \leftarrow \pi_o$;
      Convert $\widehat{\mathbf{A}}_s(t)$ by (24);
    end
    Execute the offloading decision in the network;
    Observe the next offloading-state $(\widehat{\mathbf{H}}_s(t+1))_{s\in\mathcal{S}}$;
    Get the reward $\mathfrak{R}(t)$;
    Construct and save $\langle(\widehat{\mathbf{H}}_s(t), \widehat{\mathbf{A}}_s(t), \widehat{\mathbf{H}}_s(t+1), \mathfrak{R}(t))_{s\in\mathcal{S}}\rangle$ to $\mathcal{M}_o$;
    Randomly select a mini-batch from $\mathcal{M}_o$;
    Update the parameters $\boldsymbol{\theta}_o$ and $\boldsymbol{\vartheta}_o$;
  end
end
Save this model;

quality of the generated actions. To address the limitation of SAC in handling continuous actions, the output of the actor network is transformed from a density $\pi_o(\widehat{\mathbf{A}}_s(t); \widehat{\mathbf{H}}_s(t))$ to a probability $\pi_o(\widehat{\mathbf{H}}_s(t))$. In DSAC, we employ two DNNs with parameters $\boldsymbol{\theta}_o$ and $\boldsymbol{\vartheta}_o$ to learn the policy π_o^* by maximizing both the reward and the entropy, i.e.,

$$\pi_o^* = \arg\max_{\pi_o} \mathbb{E}\left[\sum_{i=t}^{\infty} \widehat{\gamma}^{i-t}\left(\mathfrak{R}(i) + \delta F\left(\pi_o\left(\widehat{\mathbf{H}}_s(i)\right)\right)\right)\right], \tag{25}$$

where $F(\pi_o(\widehat{\mathbf{H}}_s(t))) = \mathbb{E}[-\log \pi_o(\widehat{\mathbf{H}}_s(t))]$ is the entropy. To balance the trade-off between maximizing the reward and the entropy, we introduce a temperature parameter δ. This parameter allows for adjusting the relative importance of the entropy term compared to the reward in the objective function. To maximize (25), we use soft policy iteration between policy evaluation and improvement. Let $G(\widehat{\mathbf{H}}_s(t))$ denote the soft state value function, given by

$$G(\widehat{\mathbf{H}}_s(t)) = \pi_o\left(\widehat{\mathbf{H}}_s(t)\right)^T \left[\widehat{Q}\left(\widehat{\mathbf{H}}_s(t)\right) - \delta \log \pi_o\left(\widehat{\mathbf{H}}_s(t)\right)\right], \quad \forall s \in \mathcal{S}, \forall s \in \mathcal{T}, \tag{26}$$

Algorithm 3: HDRL for JSC2O.

Input: $\mathcal{T}$, service and task information.
Output: $\mathcal{X}, \mathcal{Y}$, and the average service latency.

Load DQN and DSAC models;
Reset the mobile edge-cloud computing network;
for *time slot* $t \in \mathcal{T}$ **do**
  /* **Stage I** */
  for *edge node* $e \in \mathcal{E}$ **do**
    Get the caching-action $\mathbf{A}_e(t)$ by π_e^*;
  end
  /* **Stage II** */
  for *service* $s \in \mathcal{S}$ **do**
    Get the offloading-action $\widehat{\mathbf{A}}_s(t)$ by π_o^*;
  end
  Convert $\mathbf{A}_e(t)$ and $\widehat{\mathbf{A}}_s(t)$ into the caching decision and the offloading decision;
  Execute these two decisions in the network;
  Get the service latency $L(t)$;
end
Get the long-term average service latency by (9);

where $\widehat{Q}(\widehat{\mathbf{H}}_s(t)) : \widehat{\mathbf{H}}_s(t) \to \mathbb{R}^{|\widehat{\mathbf{A}}_s(t)|}$ is the soft Q-function with discrete form. The soft Q-function is trained to minimize the soft Bellman residual, i.e.,

$$J_{\widehat{Q}}(\boldsymbol{\theta}_o) = \mathbb{E}_{\widehat{\mathbf{H}}_s(t)\sim\mathcal{M}_o}\left[\frac{1}{2}\left(\widehat{Q}_\theta\left(\widehat{\mathbf{H}}_s(t)\right) - \hat{\widehat{Q}}\left(\widehat{\mathbf{H}}_s(t)\right)\right)^2\right], \tag{27}$$

where $\mathcal{M}_o$ is a replay memory for DSAC; $\hat{\widehat{Q}}(\cdot)$ is the target soft Q-function. The objective of the policy is given by

$$J_{\pi_o}(\boldsymbol{\vartheta}_o) = \mathbb{E}_{\widehat{\mathbf{H}}_s(t)\sim\mathcal{M}_o}\left[\pi_o\left(\widehat{\mathbf{H}}_s(t)\right)^T \left[\delta \log \pi_o\left(\widehat{\mathbf{H}}_s(t)\right) - \widehat{Q}\left(\widehat{\mathbf{H}}_s(t)\right)\right]\right]. \tag{28}$$

DSAC training at the cloud server is given in Algorithm V-C. As DSAC will utilize the caching-action information of DQN, this algorithm includes Algorithm V-B.

D. Hierarchical Deep Reinforcement Learning for JSC2O

The HDRL workflow is presented in the right side of Fig. 2. HDRL first receives physical features to make service decisions and offloading decisions, where the central agent requires action information of edge nodes. Subsequently, these decisions are applied to the mobile edge-cloud computing network environment. Finally, the environment will update the state and produces the reward. The implementation of JSC2O based on HDRL is shown in Algorithm V-C. Specifically, we first load the well-trained DQN and DSAC models by Algorithms V-B and V-C and reset the mobile edge-cloud computing network environment, steps 1-2. Then, we get the caching-action and the offloading-action according to environment information and convert them into the caching decision and the offloading decision, respectively, steps 4-10. Finally, we get the service latency by applying these two decisions in the environment, steps 11-12. We repeat the

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

above three processes, steps 3-13, and get the long-term average service latency, step 14.

E. Computational Complexity

We analyze the computational complexity of three algorithms individually. Regarding Algorithm V-B, the computational workload primarily originates from the DQN models. As the DQN models have identical network structures at each edge node, we denote the number of multiplication operations in the DNN at edge node e as Ψ. Furthermore, we use Γ and Δ to represent episode and mini-batch respectively. Therefore, the computational complexity of Algorithm V-B can be expressed as $O(\Gamma T \Delta \Psi)$. Regarding Algorithm V-C, we denote the number of multiplication operations in the DNNs at the cloud server as Φ. The computational complexity of Algorithm V-C is $O(\Gamma T \Delta E \Psi S \Phi)$. Regarding Algorithm V-C, the computational complexity can be obtained straightforwardly as $O(TE\Psi S\Phi)$. These complexity analyses provide insights into the computational requirements of each algorithm, helping understand their efficiency and scalability.

F. Performance Analysis

In this subsection, we focus on analyzing the effect on the service latency performance of the proposed framework due to the approximate optimal solution. Besides, the proposed framework based on HDRL may generate the inference error, which further affects the performance. The inference error comes from the service caching-action generated by edge agents and the computation offloading-action generated by the central agent. We can think of agent decision-making as the classification problem. If we have get the well-trained JSC2O framework, we define the inference caching error rate of edge agent e for service s as $\tau_{e,s}$, where $\tau_{e,s}$ is in [0,1]. Similarly, we define the inference offloading error rate of the central agent as $\kappa_{e,s} \in [0,1]$. We can get the following theorem.

Theorem 2: Let $L^{\text{Off}}_{e,s}(t) = \sum_{i\in\mathcal{E}} l^{tra}_{i,e,s}(t) v^i_s(1) + l^{com}_{e,s}(t) V_s(t)$, the additional service latency introduced by HDRL is $\sum_{s\in\mathcal{S}}[\sum_{e\in\mathcal{E}} \tau_{e,s} l^{tra}_{e,s}(t) + \sum_{e\in\mathcal{E}\cup\{o\}} \kappa_{e,s} L^{\text{Off}}_{e,s}(t)]$.

Proof: For simplify, we first consider the scenario in the first time slot ($t = 1$). For edge agent e and the central agent, we can have the inference caching decision and inference offloading decision, defined as

$$x^{Inf}_{e,s}(1) = \tau_{e,s} \neg x^*_{e,s}(1) + (1-\tau_{e,s}) x^*_{e,s}(1), \forall e \in \mathcal{E}, \forall s \in \mathcal{S}, \tag{29}$$

$$y^{Inf}_{e,s}(1) = \kappa_{e,s} \neg y^*_{e,s}(1) + (1-\kappa_{e,s}) y^*_{e,s}(1), \forall e \in \mathcal{E}\cup\{o\}, \quad \forall s \in \mathcal{S}, \tag{30}$$

where $x^*_{e,s}(1)$ and $y^*_{e,s}(1)$ are the optimal decisions; $\neg x^*_{e,s}(1)$ and $\neg y^*_{e,s}(1)$ are the opposite decisions, expressed as

$$\neg x^*_{e,s}(1) = \begin{cases} 1, & \text{if } x^*_{e,s}(1) = 0 \\ 0, & \text{if } x^*_{e,s}(1) = 1 \end{cases} \quad \forall e \in \mathcal{E}, \forall s \in \mathcal{S}, \tag{31}$$

$$\neg y^*_{e,s}(1) = \begin{cases} 1, & \text{if } y^*_{e,s}(1) = 0 \\ 0, & \text{if } y^*_{e,s}(1) = 1 \end{cases} \quad \forall e \in \mathcal{E}\cup\{o\}, \forall s \in \mathcal{S}. \tag{32}$$

According to (12), we can calculate the optimal service latency, written as

$$L^*(1) = \sum_{s\in\mathcal{S}} \left[\sum_{e\in\mathcal{E}} x^*_{e,s}(1) l^{tra}_{e,s}(1) + \sum_{e\in\mathcal{E}\cup\{o\}} y^*_{e,s}(1) L^{\text{Off}}_{e,s}(1) \right]. \tag{33}$$

We further have the suboptimal service latency, written as

$$L^{Inf}(1) = \sum_{s\in\mathcal{S}} \left[\sum_{e\in\mathcal{E}} x^{Inf}_{e,s}(1) l^{tra}_{e,s}(1) + \sum_{e\in\mathcal{E}\cup\{o\}} y^{Inf}_{e,s}(1) L^{\text{Off}}_{e,s}(1) \right]. \tag{34}$$

Bringing (29) and (30) into (34), we have

$$L^{Inf}(1) = \sum_{s\in\mathcal{S}} \left[\sum_{e\in\mathcal{E}} [\tau_{e,s} \neg x^*_{e,s}(1) + (1-\tau_{e,s}) x^*_{e,s}(1)] l^{tra}_{e,s}(1) + \sum_{e\in\mathcal{E}\cup\{o\}} [\kappa_{e,s} \neg y^*_{e,s}(1) + (1-\kappa_{e,s}) y^*_{e,s}(1)] \times L^{\text{Off}}_{e,s}(1) \right]. \tag{35}$$

Combining (33) and (35), we have

$$L^{Inf}(1) = \sum_{s\in\mathcal{S}} \left[\sum_{e\in\mathcal{E}} \tau_{e,s} [\neg x^*_{e,s}(1) - x^*_{e,s}(1)] l^{tra}_{e,s}(1) + \sum_{e\in\mathcal{E}\cup\{o\}} \kappa_{e,s} [\neg y^*_{e,s}(1) - y^*_{e,s}(1)] L^{\text{Off}}_{e,s}(1) \right] + L^*(1). \tag{36}$$

According to (31) and (32), (36) can be rewritten as

$$L^{Inf}(1) = \sum_{s\in\mathcal{S}} \left[\sum_{e\in\mathcal{E}} \tau_{e,s} l^{tra}_{e,s}(1) + \sum_{e\in\mathcal{E}\cup\{o\}} \kappa_{e,s} L^{\text{Off}}_{e,s}(1) \right] + L^*(1). \tag{37}$$

Therefore, the additional service latency is $\sum_{s\in\mathcal{S}} [\sum_{e\in\mathcal{E}} \tau_{e,s} l^{tra}_{e,s}(1) + \sum_{e\in\mathcal{E}\cup\{o\}} \kappa_{e,s} L^{\text{Off}}_{e,s}(1)]$. We can further extend to the scenario with multiple time slots. This completes the proof.

VI. Trace-Driven Performance Evaluation

In this section, we conduct extensive simulations to evaluate the performance of the JSC2O framework.

A. Setup and Baselines

All parameters are chosen according to a real-world scenario without sacrificing generality. Specifically, we consider a mobile edge-cloud computing network in a rectangle with a side length of 4.0 km, where nine edge nodes are empowered by heterogeneous resources and a total of eight services have heterogeneous characteristics, both following a uniform distribution. To implement HDRL, two DNNs of each edge agent consist of an input layer, an output layer, two hidden layers, and an LSTM layer. Similarly, two DNNs of the central agent comprise an input layer, an output layer, and three hidden layers. In the absence of any specified adjustments, we adopt default parameter settings for the network environment and HDRL, as provided

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PARAMETER SETTINGS

	Parameter	Value
Network Environment	T	60
	R_e	[20, 40] GB
	B_e	[100, 200] GB
	d_s	[100,200] Mbit
	C_e	[50, 100] Giga CPU cycles
	C_o	150 Giga CPU cycles
	r_s	[5,10] GB
	b_s	[10,50] GB
	$\widehat{d}_s$	[5,10] Mbit
	c_s	[0.1,0.5] Giga CPU cycle/task
	W	160 Mbps
	$\widehat{W}$	400 Mbps
HDRL	Number of neurons	128
	Optimizer	Adam
	lr of all agents	0.0001
	bs of each edge agent	64
	bs of the central agent	128
	ϵ	0.9
	γ	0.9
	δ	0.2
	$\widehat{\gamma}$	0.99

in Table III, where lr and bs are learning rate and batch size, respectively.

To enhance the practicality of our experiments, we utilized both synthetic and real datasets [38], [42]. i) Synthetic dataset: the arrival of tasks s at edge node e is the Poisson distribution with $\lambda = 5$. Table III presents the other parameters of tasks s. ii) DiDi dataset: this dataset includes the vehicular trajectories collected from Didi GAIA, which is published on Didi Chuxing's official website.[2] We extract several trajectories from a 2×2 km^2 area located in Qingyang District, Chengdu, China (30.67 N, 104.05E) on November 16, 2016, as shown in Fig. 3. We further consider three scenarios with different periods to simulate different task loads. Particularly, we define 10 seconds as one time slot. For convenience, we denote the synthetic dataset as Scenario-4. The heat maps of the four scenarios are shown in Fig. 4. As observed, the distribution of tasks is significantly imbalanced in both time and space.

For performance comparison, we implement five comparable algorithms as follows. i) Random Caching and Offloading (RCO) [20]: The caching decision and offloading decision are randomly determined. ii) Heuristic Caching and Offloading (HCO) [28]: The caching decision is determined based on the number of task types; The offloading decision is determined based on more edge execution. iii) Offloading with Fixed Caching (OFC) [43]: The service caching is disabled and the services cached at each edge node are fixed. We only optimize the offloading decision. iv) Double DQN (DDQN) [44], [45]: DDQN can eliminate the problem of overestimation. v) PPO [14]: PPO is based on policy gradient methods.

The experiments are performed on a Windows 11 desktop machine with the following specifications: 32 GB of RAM, an Intel i7 CPU, and an RTX3060-Ti GPU. The programming language is "Python 3.8" with the deep learning library "PyTorch 1.11". This setup allows for efficient execution and

[2]https://outreach.didichuxing.com/appEn-vue/dataList



Fig. 3. Trajectory heat map in Qingyang District, Chengdu, China.

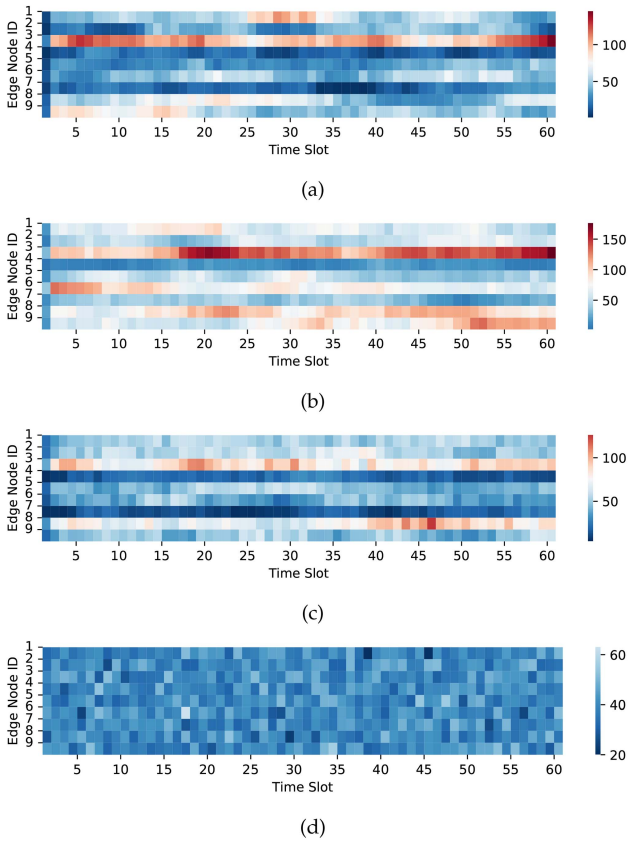


Fig. 4. Arrival of tasks at each edge node. (a) Scenario-1 with 24888 traces during 12:00-12:10; (b) Scenario-2 with 34789 traces during 15:00-15:10; (c) Scenario-3 with 25772 traces during 18:00-18:10; (d) Scenario-4 with 21825 traces.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

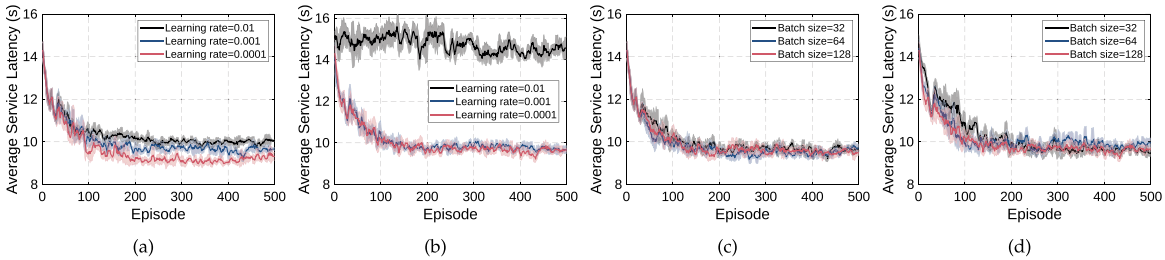


Fig. 5. Convergence under different hyper-parameters: (a) edge agents with different learning rates; (b) the central agent with different learning rates; (c) edge agents with different batch sizes; (d) the central agent with different batch size.

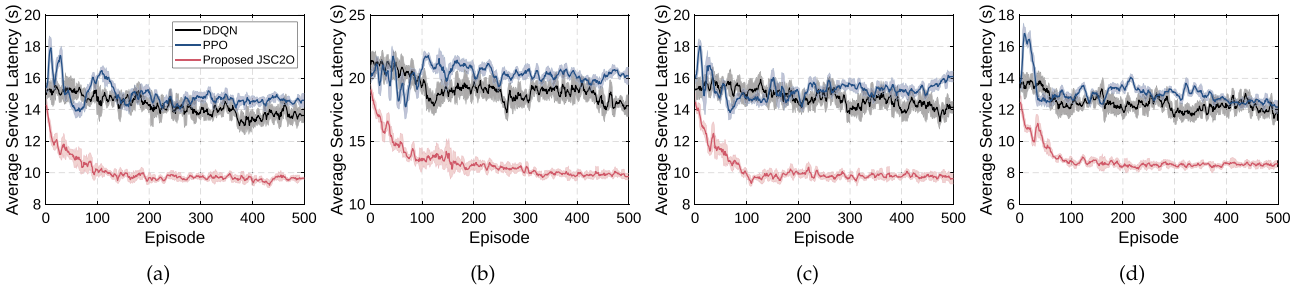


Fig. 6. Convergence of three RL-based algorithms under different scenarios: (a) Scenario-1; (b) Scenario-2; (c) Scenario-3; (d) Scenario-4.

utilization of the computational resources required for the experiments.

B. Convergence Performance

We first investigate the effects of different hyper-parameters on the convergence performance of the proposed JSC2O. Hyper-parameters of deep neural networks have an impact on training speed, generalization, and accuracy of models. In Scenario-1, we vary the values of two main parameters: learning rate and batch size as shown in Fig. 5. Learning rate controls the step size at which a neural network model updates its parameters in response to the estimated error (loss). We vary the value of learning rate for edge agents and the central agent from 0.0001 to 0.01 in Fig. 5(a) and (b), respectively. For all edge agents and the central agent, a too big learning rate cause poor convergence performance, especially in Fig. 5(b). Hence, we set the default value of all agents to 0.0001. Batch size refers to the number of training examples or data points that are processed together in a single forward/backward pass. We vary the value of batch size of edge agents and the central agent from 32 to 128 in Fig. 5(c) and (d), respectively. For edge agents, setting batch size as 64 has the best convergence performance, while the central agent with a batch size of 128 converges the fastest. We set the default values of batch size for edge agents and the central agent as 64 and 128, respectively.

We then investigate convergence performance of three RL-based algorithms (DQN, PPO, and the proposed JSC2O) under the four scenarios, as shown in Fig. 6. The results indicate that under all four scenarios, the proposed JSC2O outperforms the other two algorithms in terms of convergence speed and average service latency, achieving the lowest cost in the shortest time. Other two RL-based algorithms have slow convergence speed and high average service latency. We can see that DDQN is more effective than PPO across all four scenarios. It is worth noting that the average service latency of all three algorithms in Fig. 6(b) is highest, as Scenario-2 has the most tasks according to Fig. 4(b). Besides, JSC2O has an approximately 33% convergence improvement compared to DDQN in Fig. 6(b). Comparing Fig. 6(a), (b), and (c), we find that the proposed JSC2O has slower convergence in Scenario-1 to Scenario-3 than in Scenario-4, which may be due to the complexity of the real dataset. However, in the synthetic dataset, where there are fewer tasks, JSC2O can converge to an average service latency of around 8.4 s. Notably, PPO exhibits more volatility across all four scenarios when compared with JSC2O and DDQN.

C. Effects of Different Numbers of Edge Nodes

In this subsection, we investigate the impact of different numbers of edge nodes on average service latency across four scenarios, as depicted in Fig. 7. It is evident that the average service latency increases with an increase in the number of edge nodes for all algorithms under all scenarios. This is due to the fact that more edge nodes involve more tasks, resulting in an increase in service latency. Notably, the proposed JSC2O exhibits the lowest average service latency in all conditions, indicating that it can effectively tackle the challenges posed by dynamic tasks, heterogeneous resources, and coupled decisions to make better caching and offloading decisions. Specifically, even in complex Scenario-2, which involves the most tasks,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

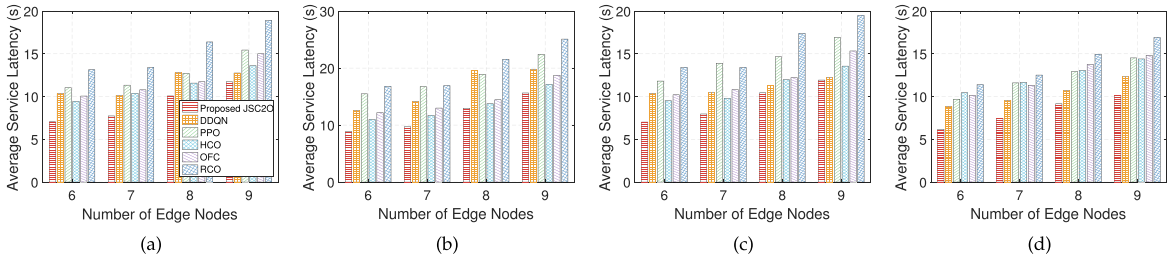


Fig. 7. Effects of different numbers of edge nodes: (a) Scenario-1; (b) Scenario-2; (c) Scenario-3; (d) Scenario-4;.

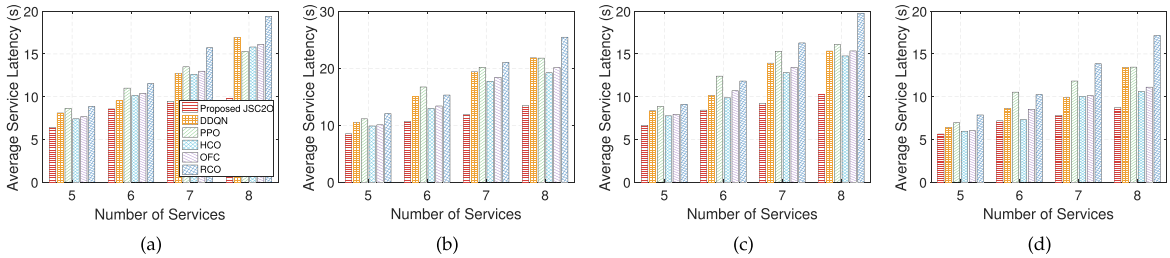


Fig. 8. Effects of different numbers of services: (a) Scenario-1; (b) Scenario-2; (c) Scenario-3; (d) Scenario-4;.

JSC2O achieves the lowest average service latency for the numbers of edge nodes presented in Fig. 7(b). When the number of edge nodes is nine, the average service latency of JSC2O is only about 15.5 s, while the other algorithms have a higher average service latency. Besides, JSC2O increases slowly as the number of edge nodes increases because JSC2O can make more reasonable and intelligent decisions about joint service caching and computation offloading.

Furthermore, in the real scenarios illustrated in Fig. 7(a), (b), and (c), it is observed that DDQN is better suited for conditions with more edge nodes than PPO. For example, in Fig. 7(a), DDQN displays a similar average service latency when the number of edge nodes is 8 or 9, potentially due to its superior edge-edge collaborative capacity, which allows it to fully leverage edge resources to offload tasks to nearby edge nodes rather than relying solely on the cloud server. Conversely, PPO exhibits unstable performance in the complex Scenario-2, as demonstrated in Fig. 7(b) and supported by Fig. 6(b). Additionally, in most real scenarios, HCO outperforms DDQN, PPO, OFC, and RCO, but its performance is comparable in the synthetic scenario presented in Fig. 7(d).

In summary, the results show that the number of edge nodes has a significant impact on the average service latency of all algorithms. JSC2O exhibits the lowest average service latency in all conditions, indicating its effectiveness in addressing the challenges posed by dynamic tasks, heterogeneous resources, and coupled decisions.

D. Effects of Different Numbers of Services

In this subsection, we aim to investigate the impact of varying numbers of services on the average service latency of the different algorithms under different scenarios. To accomplish this, we conduct experiments with four different scenarios, as depicted in Fig. 8, and measure the average service latency for different numbers of services.

As shown in Fig. 8, we observe that the average service latency increases as the number of services increases for all algorithms under all scenarios. However, we find that the proposed JSC2O has the slowest growth rate compared to other algorithms. For instance, in Fig. 8(b), as the number of services increases from 5 to 8, the average service latency of JSC2O increases only from 9 s to 13.5 s, whereas the average service latency of RCO increases dramatically from 12 s to 27 s. This suggests that varying numbers of services have a relatively small impact on JSC2O's performance, making it a more robust algorithm for dynamic and changing environments. Moreover, we find that JSC2O performs the best in Scenario-4 in terms of average service latency as shown in Fig. 8(d), where it experiences minimal changes in performance with varying numbers of services. This implies that JSC2O is better equipped to handle scenarios with high numbers of services and can maintain stable performance under such conditions.

For five baselines, it is observed that different numbers of services had significant impacts on their performance. As depicted in Fig. 8(a), when the number of services ranged from 5 to 8, the average service latency of DDQN showed a difference of around 9 s. This can be attributed to the fact that different services affect service caching, and the edge agents of DDQN are not able to make reasonable caching decisions. In contrast, when comparing DDQN and PPO in Fig. 8(a) and (d), it is found that PPO outperforms DDQN in the condition with more services. This suggests that PPO may be more suitable for caching decisions. When combined with Fig. 7(b), it can be concluded that DDQN and PPO are suitable for offloading decisions and caching decisions, respectively. Fig. 8(c) shows that our framework in a 8-service

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

scenario achieves a 39% reduction in the average service latency compared to DDQN and PPO. Finally, HCO performs better than OFC in all scenarios in terms of average service latency because OFC cannot optimize the caching decision. Additionally, the performance of RCO is the worst among the five baselines, as it has the highest average service latency in all scenarios. This can be attributed to the fact that RCO does not consider the dynamic nature of tasks and the heterogeneity of resources, resulting in suboptimal offloading decisions.

In summary, our experiments demonstrate that the number of services has a significant impact on the average service latency of different algorithms. However, JSC2O performs the best in terms of stability and reliability, making it a suitable candidate for real-world applications with varying numbers of services.

VII. Conclusion and Future Work

In this article, we have investigated the issue of joint service caching and computation offloading in mobile edge-cloud computing networks. This issue faces three key challenges of dynamic tasks, heterogeneous resources, and coupled decisions. We have formulated the optimization problem as minimizing the service latency, which is NP-hard. We have performed in-depth theoretical analyses about this problem and decomposed it into two sub-problems. We have further proposed a JSC2O based on HDRL to hierarchically solve these two sub-problems, which can effectively address these challenges. In JSC2O, the edge agents and the cloud agent collaborate to determine the caching and offloading actions, respectively. The performance of JSC2O has been meticulously evaluated through extensive trace-driven simulations on both synthetic and real datasets. The results have showcased that our proposed framework surpasses five existing baselines in terms of the average service latency. Moreover, even in complex real-world scenarios, our framework has consistently exhibited remarkable performance and effectiveness.

In future investigations, we plan to explore this issue within the context of unstable network environments. Given that the proposed framework necessitates the uploading of edge information and downloading of offloading-actions, a stable network is imperative, with uninterrupted connectivity across all edge nodes and connections. Consequently, addressing the challenge posed by network instability emerges as a meaningful research direction.

References

[1] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 22, no. 2, pp. 869–904, Second Quarter, 2020.

[2] K. Jiang, C. Sun, H. Zhou, X. Li, M. Dong, and V. C. Leung, "Intelligence-empowered mobile edge computing: Framework, issues, implementation, and outlook," *IEEE Netw.*, vol. 35, no. 5, pp. 74–82, Sep./Oct. 2021.

[3] X. Yuan, C. Sun, and S. Chen, "Cooperative DNN partitioning for accelerating DNN-empowered disease diagnosis via swarm reinforcement learning," *Appl. Soft Comput.*, vol. 148, 2023, Art. no. 110844.

[4] H. Zhou, Z. Zhang, Y. Wu, M. Dong, and V. C. Leung, "Energy efficient joint computation offloading and service caching for mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 950–961, Jun. 2023.

[5] Z. Ming, X. Li, C. Sun, Q. Fan, X. Wang, and V. C. Leung, "Dependency-aware hybrid task offloading in mobile edge computing networks," in *Proc. IEEE Int. Conf. Parallel Distrib. Syst.*, 2021, pp. 225–232.

[6] E. A. van Dis, J. Bollen, W. Zuidema, R. van Rooij, and C. L. Bockting, "ChatGPT: Five priorities for research," *Nature*, vol. 614, no. 7947, pp. 224–226, 2023.

[7] B. Yang, X. Cao, J. Bassey, X. Li, and L. Qian, "Computation offloading in multi-access edge computing: A multi-task learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 9, pp. 2745–2762, Sep. 2021.

[8] X. Wang, C. Wang, X. Li, V. C. Leung, and T. Taleb, "Federated deep reinforcement learning for Internet of Things with decentralized cooperative edge caching," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9441–9455, Oct. 2020.

[9] C. Sun, X. Wu, X. Li, Q. Fan, J. Wen, and V. C. Leung, "Cooperative computation offloading for multi-access edge computing in 6G mobile networks via soft actor critic," *IEEE Trans. Netw. Sci. Eng.*, early access, Apr. 30, 2021, doi: 10.1109/TNSE.2021.3076795.

[10] M. Xue, H. Wu, G. Peng, and K. Wolter, "DDPQN: An efficient dnn offloading strategy in local-edge-cloud collaborative environments," *IEEE Trans. Services Comput.*, vol. 15, no. 2, pp. 640–655, Mar./Apr. 2022.

[11] S. Duan et al., "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," *IEEE Commun. Surv. Tut.*, vol. 25, no. 1, pp. 591–624, First Quarter 2023.

[12] H. Zhou, K. Jiang, X. Liu, X. Li, and V. C. Leung, "Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1517–1530, Jan. 2021.

[13] B. Yang, X. Cao, C. Yuen, and L. Qian, "Offloading optimization in edge computing for deep-learning-enabled target tracking by internet of UAVs," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9878–9893, Jun. 2020.

[14] D. Ren, X. Gui, and K. Zhang, "Adaptive request scheduling and service caching for mec-assisted iot networks: An online learning approach," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17372–17386, Sep. 2022.

[15] S. Zhong, S. Guo, H. Yu, and Q. Wang, "Cooperative service caching and computation offloading in multi-access edge computing," *Comput. Netw.*, vol. 189, 2021, Art. no. 107916.

[16] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2076–2085.

[17] V. Farhadi et al., "Service placement and request scheduling for data-intensive applications in edge clouds," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 779–792, Feb. 2021.

[18] R. Zhou, X. Wu, H. Tan, and R. Zhang, "Two time-scale joint service caching and task offloading for UAV-assisted mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1189–1198.

[19] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 207–215.

[20] Z. Xue, C. Liu, C. Liao, G. Han, and Z. Sheng, "Joint service caching and computation offloading scheme based on deep reinforcement learning in vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6709–6722, May 2023.

[21] H. Zhou, Z. Wang, H. Zheng, S. He, and M. Dong, "Cost minimization-oriented computation offloading and service caching in mobile cloud-edge computing: An A3C-based approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1326–1338, May/Jun. 2023.

[22] G. Zhang, S. Zhang, W. Zhang, Z. Shen, and L. Wang, "Joint service caching, computation offloading and resource allocation in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5288–5300, Aug. 2021.

[23] W. Chu, P. Yu, Z. Yu, J. C. Lui, and Y. Lin, "Online optimal service selection, resource allocation and task offloading for multi-access edge computing: A utility-based approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4150–4167, Jul. 2023.

[24] G. Zhao, H. Xu, Y. Zhao, C. Qiao, and L. Huang, "Offloading tasks with dependency and service caching in mobile edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 11, pp. 2777–2792, Nov. 2021.

[25] Y. Zhang, M. Kishk, and S. Alouini, "Computation offloading and service caching in heterogeneous MEC wireless networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3241–3256, Jun. 2023.

[26] W. Fan et al., "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 238–256, Jan. 2024.

[27] H. Zhou, Z. Zhang, D. Li, and Z. Su, "Joint optimization of computing offloading and service caching in edge computing-based smart grid," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1122–1132, Apr.–Jun. 2023.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.

[28] X. Xu, Z. Liu, M. Bilal, S. Vimal, and H. Song, "Computation offloading and service caching for intelligent transportation systems with digital twin," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20757–20772, Nov. 2022.
[29] S.-W. Ko, S. J. Kim, H. Jung, and S. W. Choi, "Computation offloading and service caching for mobile edge computing under personalized service preference," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 6568–6583, Aug. 2022.
[30] S. Bi, L. Huang, and Y.-J. A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.
[31] Z. Lin, S. Bi, and Y.-J. A. Zhang, "Optimizing AI service placement and computation offloading in mobile edge intelligence systems," in *Proc. IEEE Glob. Commun. Conf.*, 2020, pp. 1–7.
[32] J. Li, H. Zhang, H. Ji, and X. Li, "Joint computation offloading and service caching for MEC in multi-access networks," in *Proc. IEEE 30th Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, 2019, pp. 1–6.
[33] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Joint service placement and request routing in multi-cell mobile edge computing networks," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 10–18.
[34] L. Zhang, Z. Qu, B. Ye, and B. Tang, "Joint service placement and computation offloading in mobile edge computing: An auction-based approach," in *Proc. IEEE 26th Int. Conf. Parallel Distrib. Syst.*, 2020, pp. 256–265.
[35] W. Fan et al., "Joint task offloading and service caching for multi-access edge computing in WiFi-cellular heterogeneous networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9653–9667, Nov. 2022.
[36] H. Liu, Y. Li, and S. Wang, "Request scheduling combined with load balancing in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 20841–20852, Nov. 2022.
[37] K. Peng et al., "Joint optimization of service chain caching and task offloading in mobile edge computing," *Appl. Soft Comput.*, vol. 103, 2021, Art. no. 107142.
[38] J. Ren et al., "An efficient two-layer task offloading scheme for MEC system with multiple services providers," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1519–1528.
[39] X. Wu, X. Li, J. Li, P. Ching, V. C. Leung, and H. V. Poor, "Caching transient content for IoT sensing: Multi-agent soft actor-critic," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 5886–5901, Sep. 2021.
[40] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Service placement and request routing in MEC networks with storage, computation, and communication constraints," in *Proc. IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1047–1060, Jun. 2020.
[41] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*. New York, NY, USA: Cambridge Univ. Press, 2004.
[42] X. Xu et al., "Joint task offloading and resource optimization in NOMA-based vehicular edge computing: A game-theoretic DRL approach," *J. Syst. Archit.*, vol. 134, 2023, Art. no. 102780.
[43] A. Naouri, H. Wu, N. A. Nouri, S. Dhelim, and H. Ning, "A novel framework for mobile-edge computing by optimizing task offloading," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 13065–13076, Aug. 2021.
[44] Q. Yuan, J. Li, H. Zhou, T. Lin, G. Luo, and X. Shen, "A joint service migration and mobility optimization approach for vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 9041–9052, Aug. 2020.
[45] L. Wang and G. Zhang, "Joint service caching, resource allocation and computation offloading in three-tier cooperative mobile edge computing system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3343–3353, Nov./Dec. 2023.



Chuan Sun received the BS degree from the Wuhan University of Science and Technology, Wuhan, China, in 2017 and the PhD degree from the School of Big Data & Software Engineering, Chongqing University, Chongqing, China, in 2023. He is currently a research fellow in the School of Computer Science and Engineering, Nanyang Technological University, 50 Nanyang Avenue, Singapore. His current research interests include federated learning, reinforcement learning, and mobile edge computing.



Xiuhua Li (Member, IEEE) received the BS degree from the Honors School, Harbin Institute of Technology, China, in 2011, the MS degree from the School of Electronics and Information Engineering, Harbin Institute of Technology, in 2013, and the PhD degree from the Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, Canada, in 2018. He is currently a tenure-track Assistant Professor with the School of Big Data & Software Engineering and the Dean of the Institute of Intelligent Software and Services Computing associated with the Key Laboratory of Dependable Service Computing in Cyber Physical Society (Chongqing University), Education Ministry, China. His current research interests are 5G/6G mobile Internet, mobile edge computing and caching, Big Data analytics, and machine learning.



Chenyang Wang (Member, IEEE) received the BS and MS degrees in computer science and technology from Henan Normal University, Xinxiang, China, in 2013 and 2017, respectively, and the PhD degree from the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University, Tianjin, China, in 2023. He is also a visiting PhD student under the support of the China Scholarship Council (CSC) with the School of Electrical Engineering, Aalto University, Espoo, Finland since 15 May 2021. His current research interests include edge computing, Big Data analytics, reinforcement learning, and deep learning.



Qiang He (Senior Member, IEEE) received the first PhD degree from the Swinburne University of Technology, Australia, in 2009, and the second PhD degree in computer science and engineering from the Huazhong University of Science and Technology, China, in 2010. He is an associate professor of the Swinburne University of Technology. His research interests include mobile edge computing, software engineering, service computing, and cloud computing. More details about his research can be found at https://sites.google.com/site/heqiang/.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.



Xiaofei Wang (Senior Member, IEEE) received the BS degree from the Huazhong University of Science and Technology, China, and the MS and PhD degrees from Seoul National University, Seoul, South Korea. He was a postdoctoral fellow with the University of British Columbia, Vancouver, Canada, from 2014 to 2016. He is currently a professor with the College of Intelligence and Computing, Tianjin University, Tianjin, China. Focusing on the research of edge computing, edge intelligence, and edge systems, he has published more than 150 technical papers in IEEE JSAC, TCC, ToN, TWC, IoTJ, COMST, TMM, INFOCOM, ICDCS and so on. In 2017, he was the recipient of the "IEEE ComSoc Fred W. Ellersick Prize", and in 2022, he received the "IEEE ComSoc Asia-Pacific Outstanding Paper Award".



Victor C. M. Leung (Life Fellow, IEEE) is a distinguished professor of computer science and software engineering with Shenzhen University, China. He is also an emeritus professor of electrical and computer engineering and director of the Laboratory for Wireless Networks and Mobile Systems with the University of British Columbia (UBC), Canada. His research is in the broad areas of wireless networks and mobile systems, and he has published widely in these areas. His published works have together attracted more than 60,000 citations. He is named in the current Clarivate Analytics list of "Highly Cited Researchers". He is serving on the editorial boards of the *IEEE Transactions on Green Communications and Networking*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Computational Social Systems*, and several other journals. He received the 1977 APEBC Gold Medal, 1977-1981 NSERC Postgraduate Scholarships, IEEE Vancouver Section Centennial Award, 2011 UBC Killam Research Prize, 2017 Canadian Award for Telecommunications Research, 2018 IEEE TCGCC Distinguished Technical Achievement Recognition Award, and 2018 ACM MSWiM Reginald Fessenden Award. He co-authored papers that won the 2017 IEEE ComSoc Fred W. Ellersick Prize, 2017 IEEE Systems Journal Best Paper Award, 2018 IEEE CSIM Best Journal Paper Award, and 2019 IEEE TCGCC Best Journal Paper Award. He is a Fellow of the Royal Society of Canada (Academy of Science), Canadian Academy of Engineering, and Engineering Institute of Canada.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 10:13:42 UTC from IEEE Xplore. Restrictions apply.